

ISM2411 — Lab Week 01

Computer Vocabulary & File System Tour — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 01 · Unit 1 · Foundations

Contents

1	Session Snapshot	2
2	Learning Objectives	2
3	Before Class — Setup Checklist	3
4	Materials Needed	3
5	Timing Plan (75 minutes)	3
6	Segment-by-Segment Walkthrough	5
6.1	Intro (0:00–0:05)	5
6.2	Exercise 1 — Trace a Real File Path (0:05–0:13, 8 min)	5
6.3	Exercise 2 — Map Your Project Folder (0:13–0:19, 6 min)	6
6.4	Exercise 3 — Vocabulary Check (0:19–0:34, 15 min)	8
6.5	Exercise 4 — Automation Reflection (0:34–0:44, 10 min, pair discussion)	9
6.6	Exercise 5 — Draw the Directory Tree (0:44–0:56, 12 min)	11
6.7	Exercise 6 — RAM vs. Storage Scenarios (0:56–1:06, 10 min)	13
6.8	Stretch A & B (as time allows, folded into wrap-up)	14
7	Wrap-Up (last ~9 minutes of the 1:06–1:15 block)	15
8	Appendix A — Reference Answer Key	15
9	Appendix B — Extra Practice (only if the class finishes early)	16

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 01 Lab — Computer Vocabulary & File System Tour
Unit	Unit 1 · Foundations
Class length	75 minutes
Format	No coding today — a discussion- and worksheet-driven session. Students work on paper/in a doc, in pairs for Exercise 4, individually for the rest
Prerequisites	None — this is the first lab of the semester
Student-facing lab page	markumreed.github.io/ism2411 — week01_lab
Exercises covered	Exercises 1–6 (required) + Stretch A/B (as time allows)
Submission	No Canvas code submission this week — students bring their vocabulary answers, folder screenshot, and Exercise 4 automation examples to class

This is the only lab all semester with zero code in it, and that is deliberate: everything from Module 02 onward assumes students already have a rock-solid mental model of what a file *is*, where it *lives*, and how a computer *finds* it. Because there is no code to anchor the room’s attention, this session lives or dies on your pacing and your questions — the content itself (nine vocabulary words, three RAM/storage scenarios) is thin enough to blow through in fifteen minutes if you let it. Don’t. This guide over-provisions discussion prompts and extra worked examples specifically so you have material to slow down with.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Read any absolute file path and correctly label its root, directory chain, filename, and extension.
2. Explain, in their own words, what CPU, RAM, storage, file, directory, path, extension, program, and interpreter each mean — and how they relate to each other as one system.
3. Draw the directory tree implied by a set of file paths, and write a correct relative path between two files in that tree.
4. Classify a real-world “something went wrong” scenario as a RAM problem, a storage problem, or neither.

5. Identify a repetitive spreadsheet task as a plausible automation candidate — without yet knowing how to build the automation.

3 Before Class — Setup Checklist

- ☐ No IDE needed today. Have Finder (Mac) / File Explorer (Windows) ready to demo on the projector for Exercise 1.
- ☐ Bring (or project) a blank document / whiteboard for building the directory tree live in Exercise 5.
- ☐ Decide your pairing strategy for Exercise 4 in advance (adjacent seats, or count-off) — don't lose two minutes to pairing logistics mid-class.
- ☐ Pre-read the nine vocabulary terms below and pick one *wrong-but-plausible* definition per term to float to the class and have them correct (this is the single best technique for making a vocab-recitation exercise feel like a real class instead of a spelling test — see Exercise 3).
- ☐ Print or project the three given file paths for Exercise 5 and the three RAM/storage scenarios for Exercise 6 so students aren't squinting at a shared screen from the back row.

4 Materials Needed

- Projector/screen-share with Finder/Explorer visible
- Students: laptop (file system only, no code editor needed) or paper
- No Python, no internet research required for the core exercises (Stretch B needs a web search)

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:05	Welcome, framing: “no code today, and here’s why that’s not a break”	5
0:05–0:13	Exercise 1 — Trace a real file path	8
0:13–0:19	Exercise 2 — Build the <code>ism2411/</code> project folder	6
0:19–0:34	Exercise 3 — Nine vocabulary terms	15
0:34–0:44	Exercise 4 — Automation reflection (pair discussion)	10

Time	Segment	Minutes
0:44–0:56	Exercise 5 — Draw the directory tree (given + one extra rep)	12
0:56–1:06	Exercise 6 — RAM vs. storage (three given + two extra scenarios)	10
1:06–1:15	Stretch A/B preview + wrap-up, reflection, submission checklist	9

This lab’s raw content (six exercises, no code) does not naturally fill 75 minutes at a lecture pace — it fills 75 minutes only if you run it as a genuine discussion with cold-calling, pair-share, and the extra reps built into this guide (marked **EXTRA** below). Do not compress the schedule by skipping the extras; skip Stretch A/B instead if you’re short on time, since those are explicitly designed as optional take-home material.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:05)

Say to the class:

“Welcome to ISM2411. Today has zero code in it, and I want to tell you why before you decide this is a throwaway class. Every single thing we do for the rest of the semester — every script, every file you read or write, every error message about a file ‘not found’ — depends on you having an automatic, no-longer-consciously-thought-about understanding of what a file is, where it lives, and how a program finds it. If that model is shaky, every future module gets harder than it needs to be. So today we build that model, on paper, no distractions from syntax.”

Do: Write today’s six exercises on the board as a checklist (or leave the lab page projected in a corner of the screen) so the room can track progress visually — with no code running, students lose the sense of “are we making progress” that a working script normally provides. This checklist substitutes for that.

6.2 Exercise 1 — Trace a Real File Path (0:05–0:13, 8 min)

Teaching goal: Anchor the abstract idea of “a path” in something concretely on the student’s own machine, before moving to hypothetical examples.

Say to the class:

“Open Finder or File Explorer right now. Go to your Downloads folder — everyone has files in there. Pick any one file, right-click it, and choose Get Info on a Mac or Properties on Windows. There’s a full path shown there. Copy it down.”

Do, live on the projector: Demo this once yourself first so students see exactly which menu item to click (this is the single most common place to lose five minutes — half the room will click the wrong thing if you don’t show it). Then walk your own example path on screen:

`/Users/yourname/Downloads/syllabus.pdf`

Label it out loud, pointing at each segment as you say it:

- **Root** — `/` — the very top of the entire filesystem. On Windows this is a drive letter like `C:\` instead, worth mentioning explicitly since roughly half the room is on Windows.
- **Directory chain** — `Users/yourname/Downloads/` — the sequence of folders you pass through to get to the file.
- **Filename** — `syllabus`
- **Extension** — `.pdf` — tells the operating system (and every application) what *kind* of file this

is and which program should open it by default.

Have every student write their own labeled path (their own Downloads file, not yours) — this individual repetition, immediately after your demo, is what makes the concept land; watching you do it once is not enough.

Common student confusion to watch for:

- Windows students writing `C:\Users\name\Downloads\file.pdf` and being unsure whether `C:\` counts as “the root” the same way `/` does on Mac — confirm explicitly that yes, it’s the Windows equivalent, and the backslash-vs-forward-slash difference is a genuine platform difference they’ll need to remember.
- Students who copy a path that has no extension (e.g. a folder, or a file with no visible extension because their OS hides known extensions by default) — if you see this, pause and use it as a teaching moment: ask the room “why might an OS choose to hide extensions from users by default, and what problem can that cause?” (Answer: it’s meant to look cleaner for non-technical users, but it means people can’t tell a real `.pdf` from a disguised `invoice.pdf.exe` — a real security angle worth 30 seconds.)

Check for understanding: Cold-call 2–3 students to read their labeled path out loud. Listen specifically for whether they say “directory chain” as a chain of *nested* folders (not just a flat list) — that nesting concept is the entire foundation for Exercise 5.

6.3 Exercise 2 — Map Your Project Folder (0:13–0:19, 6 min)

Teaching goal: A real, physical artifact (a folder on their machine) that every future assignment will live inside — build it once, correctly, and it removes an entire category of “where did I save that” friction all semester.

Say to the class:

“This folder is going to hold every assignment you submit this semester. Get the structure right today and you’ll never think about it again.”

Do: Have every student create, live, right now:

```
ism2411/  
├─ unit1/  
├─ unit2/  
├─ unit3/  
└─ ...
```

Walk the room while they do this — this is a good moment to check laptops directly rather than relying on a show of hands, since “I made the folder” and “I made the folder in the right place, spelled correctly” are different claims.

Common student mistakes to watch for:

- Creating the folder inside another already-messy folder (e.g. nested three levels deep inside Downloads) rather than somewhere sensible like Documents or Desktop — not wrong, exactly, but worth a gentle nudge toward a location they'll actually remember in Week 10.
- Spelling `ism2411` inconsistently (capital letters, spaces, `ISM 2411` with a space) — flag that consistent, no-space, lowercase naming will matter a lot once they're typing folder names into scripts starting in a few weeks, where a space in a path causes real errors.

Check for understanding: Walk by and visually confirm five or six laptops show the folder open in Finder/Explorer with subfolders visible — this is a “verify by looking,” not a “verify by asking,” exercise.

6.4 Exercise 3 — Vocabulary Check (0:19–0:34, 15 min)

Teaching goal: Nine terms that form one connected mental model, not nine disconnected flash-cards — the goal is for students to be able to explain *how the terms relate*, not just recite definitions.

Say to the class:

“Nine terms. I’m not going to just read you definitions — for each one, I want a volunteer to take a first attempt, out loud, before I confirm or correct it. Getting it slightly wrong out loud and then hearing the correction is how this actually sticks.”

Do — work through each term as a mini-dialogue, not a lecture. For each term below: ask for a volunteer definition first, then confirm/refine using the reference definition, then use the one-line “why it matters” hook to connect it to the next term.

Term	Reference definition	Connects to
CPU	The chip that executes your program’s instructions, one at a time.	“Executes” — but from where does it read those instructions?
RAM	Fast, temporary memory where running programs and open files live; clears when the computer shuts down.	The CPU pulls from RAM constantly — it’s the workspace.
Storage	Slower, permanent memory where files live even when the computer is off.	RAM’s contents come <i>from</i> storage when a program starts.
File	A named sequence of bytes stored on disk.	Files live <i>in</i> directories — never floating alone.
Directory	A container for files and other directories; also called a folder.	Directories nest — a directory can contain other directories.
Path	The full address of a file within the directory hierarchy.	This is Exercise 1’s labeled string, formalized.
Extension	The suffix after the dot in a filename that tells the OS/apps what kind of file it is.	The last segment of a path — connects back to Exercise 1.
Program	A set of instructions that tells the computer what to do.	Programs are themselves just files, until they’re running.

Term	Reference definition	Connects to
Interpreter	The program that reads and executes another program's instructions, one line at a time — Python's interpreter, specifically, for this course.	This is <i>what runs</i> every script you'll write starting Module 02.

The connective narrative to say out loud once you've covered all nine (this is the payoff of the table above — do not skip it):

“Here’s the whole story in one breath: your files live in storage, organized into nested directories, each reachable by a path. When you run a program, the interpreter reads that program’s instructions from storage and the CPU executes them — and while it’s running, everything it’s actively working with sits in RAM, not storage, which is why unsaved work disappears if the power cuts. That’s the entire mental model. Every module this semester is a variation on some piece of that sentence.”

Have students write their own one-sentence definitions for all nine, in their own words, individually — this is the exercise’s actual required deliverable; the group discussion above should happen *before* they write, not instead of writing.

Common student mistakes to watch for:

- Confusing “program” and “process” (a process is a program that’s currently running) — not required vocabulary this week, but if a sharp student raises it, it’s a good preview hook for later.
- Writing a definition of RAM or storage that’s just a memorized textbook sentence rather than their own phrasing — if a definition sounds identical to the reading, ask them to explain it to you as if to a friend with zero CS background, and have them write *that* version down instead.

Check for understanding: Ask the room to raise a hand if they can now explain, without looking at notes, why unsaved work disappears when a laptop dies — but *not* why a missing file causes a “file not found” error (these are two different failure modes, RAM vs. storage, and conflating them is the single most common misconception this exercise is designed to correct — it’s also exactly what Exercise 6 tests).

6.5 Exercise 4 — Automation Reflection (0:34–0:44, 10 min, pair discussion)

Teaching goal: Get students recognizing *automatable patterns* in tasks they already do, without expecting them to know how to build the automation yet — this is a motivation exercise, not a skills exercise.

Say to the class:

“Turn to the person next to you. Each of you names three things you currently do in Excel — or any repetitive task, doesn’t have to be Excel — that take more than ten minutes. For each one, discuss: could a program that runs in a few seconds do this instead? You don’t need to know how yet — just whether the task has the right shape.”

Do: Give pairs 6 minutes to discuss and write down three tasks each (or three shared ones), then use the last 4 minutes to have 2–3 pairs share one example each with the whole room. Seed the room with these examples if a pair is stuck:

1. Manually copying last week’s sales numbers into a summary tab every Monday morning.
2. Reformatting a CSV export from another system so the columns match a template.
3. Cross-checking two spreadsheets by eye to find rows that don’t match.

The distinguishing question to drill into during share-out: “What makes this task the *kind of thing* a computer is good at?” Look for answers that mention **repetition** (the same steps, over and over) and **clear rules** (a human doesn’t have to use judgment at each step) — those two properties are exactly what make a task automatable, and naming them explicitly is the actual learning goal, more than the specific examples themselves.

Common student mistakes to watch for:

- Naming a task that genuinely requires human judgment at every step (e.g., “deciding which client to prioritize this week”) — this is a good moment to contrast with the automatable examples: judgment-heavy tasks are *harder* to automate, and that distinction is worth surfacing rather than correcting away.

Check for understanding: During share-out, ask the presenting pair: “If I gave you unlimited time to learn Python, which of your three tasks would you tackle first, and why?” A good answer names the most repetitive, rule-based one — not necessarily the most time-consuming one.

6.6 Exercise 5 — Draw the Directory Tree (0:44–0:56, 12 min)

Teaching goal: The reverse direction of Exercise 1 — given paths, reconstruct the tree they imply, including the idea that shared path prefixes mean shared parent folders.

Say to the class:

“Exercise 1 went from a real folder to a written path. Now we go backwards: I give you paths, you draw the tree.”

Do, live on the board/screen. Given:

```
/home/alice/projects/q1_report/data/sales.csv
/home/alice/projects/q2_report/data/sales.csv
/home/alice/projects/q1_report/scripts/summary.py
```

Build the tree incrementally, one path at a time, rather than presenting the finished tree — this is the part that actually teaches the merging logic:

1. First path alone: a straight chain, home → alice → projects → q1_report → data → sales.csv.
2. Second path: ask the room “where does this branch off from the first one?” — the answer is at projects/, since both paths share home/alice/projects/ and diverge at q1_report vs q2_report.
3. Third path: ask again — it shares projects/q1_report/ with the first path and diverges at data vs scripts.

Finished tree:

```
/home/alice/projects/
├── q1_report/
│   ├── data/
│   │   └── sales.csv
│   └── scripts/
│       └── summary.py
└── q2_report/
    └── data/
        └── sales.csv
```

Then have students write one relative path, from scripts/summary.py to q2_report/data/sales.csv:

```
../../q2_report/data/sales.csv
```

Walk it out loud: “Starting in scripts/, one .. goes up to q1_report/, a second .. goes up to projects/, then back down through q2_report/data/ to sales.csv.”

EXTRA — a second rep with different paths, for the room to work independently (do this if the first tree took less than 8 minutes, or assign it as the last two minutes of independent practice)

regardless — a single rep of tree-drawing is thin for a 12-minute block):

```
/home/bob/clients/acme/contracts/master.docx
/home/bob/clients/acme/invoices/2024/jan.pdf
/home/bob/clients/globex/contracts/master.docx
```

Have students draw this tree on their own (it branches at `clients/`, into `acme/` and `globex/`, and `acme/` itself branches again into `contracts/` and `invoices/2024/`), then write the relative path from `contracts/master.docx` to `invoices/2024/jan.pdf` within the `acme` branch:

```
../invoices/2024/jan.pdf
```

(Only one `..` needed this time, since `contracts/` and `invoices/2024/` share the same immediate parent, `acme/` — a good contrast with the first example’s two `..`s, and worth asking the room to explain *why* this one only needs one.)

Common student mistakes to watch for:

- Drawing two separate trees for `q1_report` and `q2_report` instead of recognizing they share `projects/` as a common parent — this is the core misconception the exercise targets; if you see it, ask “do both of these paths pass through the exact same folder at some point?” and let them find `projects/` themselves.
- Getting the relative path direction backwards (writing a path *from* the destination *to* the start) — have them re-read the instruction “from `scripts/summary.py` to `q2_report/...`” and confirm which file they’re starting *in*.

Check for understanding: “If I added a fourth file, `/home/alice/projects/q1_report/data/summary.csv`, where does it attach to the tree we already drew?” (Under `data/`, alongside `sales.csv` — a quick, low-stakes way to confirm the merging logic generalizes.)

6.7 Exercise 6 — RAM vs. Storage Scenarios (0:56–1:06, 10 min)

Teaching goal: Apply the RAM/storage distinction from Exercise 3 to diagnose real (if simplified) technical situations — this is the exercise that reveals whether Exercise 3’s vocabulary actually became usable knowledge or stayed inert facts.

Say to the class:

“Three scenarios. For each: is this a RAM problem, a storage problem, or something else? Say your reasoning, not just the label.”

Do — work through all three together, cold-calling for the reasoning before you confirm:

(a) You edited a spreadsheet for an hour and your laptop died. When you restart, your changes are gone. — **RAM**. The unsaved edits existed only in fast, temporary memory. Since they were never written to storage (never saved), losing power erased them completely.

(b) You try to open a CSV file and get an error saying the file cannot be found. — **Storage**. Either the path is wrong or the file doesn’t exist where you’re looking — this has nothing to do with RAM.

(c) Your Python script is running slowly when processing a 2 GB file. — **Storage I/O or RAM capacity, or both**. Reading 2 GB off disk takes real time, and if the machine doesn’t have enough RAM to hold the whole file while processing it, the OS has to shuffle data back and forth between RAM and storage — much slower than working entirely in RAM.

EXTRA — two more scenarios, to use if the room handles the first three quickly, or to hold in reserve for the wrap-up if you need one more rep:

(d) You close your code editor without saving, then reopen it later and your work is still there, exactly as you left it. — **Neither, really — worth discussing as a trick**. Most modern editors auto-save to a temp file in *storage* in the background, specifically to survive exactly this scenario; ask the room “so was this actually a RAM problem you got lucky on, or never a RAM problem at all?” (It was never purely a RAM problem — the editor already wrote it to storage on your behalf, which is precisely why it’s still there.)

(e) A coworker emails you a spreadsheet, you open it, edit a few cells, and email it right back without ever clicking “Save As” to a new location. — **Neither RAM nor storage in the failure sense — this one has no failure, which is itself worth naming**. Use it to check whether students can identify when a scenario *isn’t* actually diagnosing a RAM/storage problem at all, which is as important a skill as correctly diagnosing the ones that are.

Common student mistakes to watch for:

- Labeling scenario (c) as purely a “RAM problem” or purely a “storage problem” instead of recognizing it can be either, or both, depending on the machine — press for the *conditional* reasoning (“it depends on whether...”) rather than accepting a flat one-word answer.
- For scenario (b), guessing “RAM” because the error happened “while the program was running” — redirect: the *error itself* is about a missing file on disk, which is a storage concept

regardless of when in the program’s execution it surfaces.

Check for understanding: “Someone tell me, in one sentence, the general rule for telling RAM problems and storage problems apart.” (Something like: “If it’s about something being lost when the power went out, it’s RAM. If it’s about a file not existing or being slow to read, it’s storage.”)

6.8 Stretch A & B (as time allows, folded into wrap-up)

Stretch A — Read an unfamiliar .py file. Frame it for take-home practice rather than in-class time: “Find any short Python script online, 5–15 lines, and write one sentence per line explaining what it does — even if you’re guessing on some of it. You’ll do this constantly in your career, long before you can write everything from scratch.” No live demo needed today since no Python has been introduced yet; this is intentionally a low-stakes first exposure.

Stretch B — Research: how much RAM does a real dataset need? “Look up how much RAM Python needs to hold a pandas DataFrame with 1 million rows and 10 float columns, and write 3–4 sentences on why that matters for analysts.” If a student asks for the number during class: a rough rule of thumb is that a pandas float64 DataFrame uses about 8 bytes per cell, so $1,000,000 \text{ rows} \times 10 \text{ columns} \times 8 \text{ bytes} \approx 80 \text{ MB}$ for the raw data alone — but pandas overhead, indexes, and any object/string columns can push real usage well above that back-of-envelope number, which is exactly the kind of nuance a good Stretch B answer should surface rather than just quoting a single figure.

7 Wrap-Up (last ~9 minutes of the 1:06–1:15 block)

Review the reflection questions out loud (full text on the student lab page) — preview what a strong answer looks like without answering for them:

1. *How would you have explained opening an Excel file before today?* — There’s no wrong answer; the point is noticing the gap between a vague “it just opens” and today’s model (interpreter/program reads instructions, files come from storage into RAM). If the room struggles to articulate a “before,” that’s fine — say so and move on.
2. *Which of the nine terms are you least confident about?* — Genuinely useful information for you as the instructor; if several students name the same term, make a mental note to revisit it briefly at the start of Module 02.
3. *What would you need to learn to actually build one of your Exercise 4 automations?* — Accept “learn Python,” but push for something more specific if you have time: “learn how to read a spreadsheet file into a program” is a much better answer than just naming the language, and previews exactly what Module 02 starts doing.

Review the submission checklist together:

- ☐ Written, labeled file path from Exercise 1 (root, directory chain, filename, extension)
- ☐ ism2411/ folder with one subfolder per unit, visible in Finder/Explorer
- ☐ All nine vocabulary terms defined in the student’s own words
- ☐ Three automation examples from Exercise 4, with explanations
- ☐ Directory tree drawn from Exercise 5’s three paths, plus the relative path
- ☐ All three RAM-vs-storage scenarios from Exercise 6 answered with reasoning
- ☐ Brought to class — no Canvas code submission this week

Preview Module 02: “Next time, everything we talked about today in the abstract — files, paths, directories — becomes real. You’ll open a terminal, navigate the file system by typing instead of clicking, and write and run your very first Python script. The vocabulary from today is what makes next week’s terminal commands make sense instead of feeling like magic incantations.”

8 Appendix A — Reference Answer Key

Exercise 1 (example). /Users/yourname/Downloads/syllabus.pdf → Root: /. Directory chain: Users → yourname → Downloads. Filename: syllabus. Extension: .pdf.

Exercise 3 (reference one-liners — students should rephrase in their own words):

- **CPU** — the chip that executes a program’s instructions, one at a time.
- **RAM** — fast, temporary memory where running programs and open files live; clears on shut-down.
- **Storage** — slower, permanent memory where files persist even when the computer is off.
- **File** — a named sequence of bytes stored on disk.
- **Directory** — a container for files and other directories.

- **Path** — the full address of a file within the directory hierarchy.
- **Extension** — the suffix after the dot that tells the OS/apps what kind of file it is.
- **Program** — a set of instructions telling the computer what to do.
- **Interpreter** — the program that reads and executes another program's instructions, one line at a time.

Exercise 5 (given paths):

```
/home/alice/projects/
├── q1_report/
│   ├── data/
│   │   └── sales.csv
│   └── scripts/
│       └── summary.py
└── q2_report/
    └── data/
        └── sales.csv
```

Relative path, scripts/summary.py → q2_report/data/sales.csv: ../../q2_report/data/sales.csv

Exercise 5 (EXTRA paths):

```
/home/bob/clients/
├── acme/
│   ├── contracts/
│   │   └── master.docx
│   └── invoices/
│       └── 2024/
│           └── jan.pdf
└── globex/
    ├── contracts/
    │   └── master.docx
```

Relative path, contracts/master.docx → invoices/2024/jan.pdf: ../invoices/2024/jan.pdf

Exercise 6: (a) RAM — unsaved changes lost on power loss. (b) Storage — wrong or missing path. (c) Storage I/O and/or RAM capacity. **EXTRA:** (d) Neither — most editors auto-save to storage in the background. (e) Neither — no failure occurred; the round trip never depended on local storage at all.

9 Appendix B — Extra Practice (only if the class finishes early)

This lab is naturally thin on graded content, so two extras (Exercise 5's second tree, Exercise 6's two extra scenarios) are already built into the main timing plan above. If a section still finishes early after those, use this additional round:

Extra — one more directory tree, individually or in pairs:


```
/srv/media/library/movies/2020/inception.mkv  
/srv/media/library/movies/2010/inception.mkv  
/srv/media/library/shows/friends/s01e01.mkv
```

Have students draw the tree (branches at library/ into movies/ and shows/; movies/ branches again into 2020/ and 2010/) and write the relative path from movies/2010/inception.mkv to shows/friends/s01e01.mkv: ../../shows/friends/s01e01.mkv. This is a good closer because the two inception.mkv files having the *same filename in different folders* is itself worth a 20-second aside: filenames only have to be unique *within* a directory, not across the whole filesystem — a small but genuinely useful fact that resolves a common point of confusion later in the semester.